Introduzione

Il CareGiverBot (CGB) ha lo scopo di aiutare gli anziani che hanno bisogno di sostegno in tutti i momenti del giorno e potrà essere utilizzato nelle case di riposo per facilitare il lavoro degli operatori. Il CGB farà da supporto riproducendo, in base a quello che ascolta, le registrazioni di chi si occupa degli anziani. Questo sistema è composto da una scheda Raspberry compresa di microfono e cassa, inoltre se connesso alla rete sono disponibili anche un'interfaccia web e feedback in tempo reale tramite Telegram.

Cosa fa in breve

La principale funzione del CGB è rispondere a domande o affermazioni tramite la riproduzione di registrazioni effettuate precedentemente. Quando il sistema è in ascolto effettua un'analisi delle parole che riesce a comprendere, e in base ad esse sceglie la risposta da riprodurre più opportuna.
Nel caso in cui l'audio analizzato non trovi una corrispondenza con le risposte che sono state inserite nel sistema, il CGB provvederà ad inviare un messaggio all'operatore attraverso l'app Telegram. L'operatore ha la possibilità di rispondere direttamente alla richiesta tramite un messaggio vocale, che verrà riprodotto dal raspberry. Più nel dettaglio il sistema riproduce i messaggi che riceve attraverso Telegram in maniera proattiva, utile per rispondere a domande particolari dell'anziano o fare avvisi.
Il CareGiverBot può essere gestito tramite un'interfaccia web, utile per la modifica delle impostazioni della funzione sveglia, per il collegamento alla chat Telegram e la gestione delle risposte che il CGB riproduce. Per svolgere le sue funzioni di assistenza tramite Telegram è fondamentale che il CGB sia connesso ad Internet, mentre per l'interfaccia web è necessario che sia collegato alla rete locale.

Più nel dettaglio

La scelta delle risposte avviene tramite la valutazione delle parole all'interno della richiesta. Una volta ricevuta la frase dalla IA si analizzano le parole più lunghe di 3 caratteri, così da ignorare gli articoli, per ogni parola si ricerca nel DB una corrispondenza tra le parole chiave e relative risposte inserite. Se è presente una risposta con la parola chiave ricercata allora l'id della risposta viene inserito in una lista. Completata l'analisi della domanda, si procede a valutare la risposta più probabile: ogni id può essere presente più volte, in base alle parole chiave, l'id con frequenza maggiore viene considerato quello corretto. Se c'è una parità allora il sistema si comporta come se non avesse compreso la domanda, inviando un messaggio tramite il bot telegram con relativa registrazione.

Il framework Django gestisce le risposte che il CGB è in grado di fornire, si occupa della comunicazione con il DB locale e della loro rappresentazione. Django utilizza una classe che viene chiamata **Model** per rappresentare gli oggetti che vengono inseriti nel DB:

```
class Risposta(models.Model):
    def __str__(self):
        return self.nome
    idr = models.BigAutoField(primary_key=True)
    nome = models.CharField(max_length=150)
    percorsoFile = models.CharField(max_length=150)


class Keyword(models.Model):
    def __str__(self):
        return self.keyword
    id = models.BigAutoField(primary_key=True)
    keyword = models.CharField(max_length=150)


class Relazione(models.Model):
    idRisposta = models.ForeignKey(Risposta, on_delete=models.CASCADE)
    idKeyword = models.ForeignKey(Keyword, on_delete=models.CASCADE)
```

Si creano sotto classi di Model per specificare i campi che serviranno per le tabelle nel DB, questi vengono poi tradotti in comandi SQL dal framework. Nel caso avvengano dei cambiamenti è necessario informare il DB per aggiornare le tabelle, per fare ciò si utilizzano i comandi:
python manage.py makemigrations ***Malphite***
python manage.py sqlmigrate ***Malphite 0001***
Nella quale **Malphite** corrisponde al nome dell'applicazione all'interno di Django e **0001** corrisponde al numero di migrazione, è specificato dall'output del primo comando.

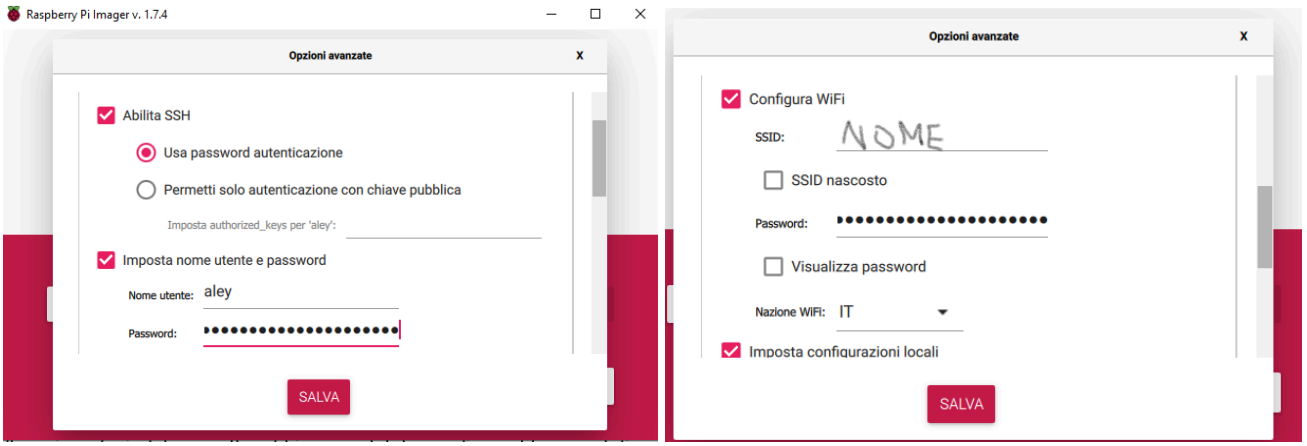
La modalità principale del CGB è quella **attiva**, essa consiste nell'ascolto attivo di domande, creazione della traduzione e ricerca di risposta. Per poter creare un modello Speech-to-text abbiamo creato una modalità **passiva**, attualmente è costruita per facilitare i test ma il suo scopo ultimo è quello di registrare frasi degli anziani, da utilizzare come base per la creazione di un modello STT tramite l'addestramento della IA. Essendo una parte secondaria abbiamo sviluppato questa modalità per velocizzare l'acquisizione di registrazione e relativa traduzione: una volta avviato il sistema con la modalità passiva viene richiesta la frase che verrà registrata, successivamente verrà avviato un loop di registrazioni, fino all'utilizzo di CTRL+C. Per ogni registrazione il file CSV viene aggiornato (o creato se non è presente) con il nuovo file audio, dimensione in byte e la frase inserita inizialmente.

Linguaggi e tecnologie utilizzati

Di seguito ci sono i principali strumenti utilizzati all'interno del CGB:
Python - Abbiamo deciso di utilizzare il linguaggio Python per la sua elasticità e portabilità ma, principalmente, perché le principali tecnologie che siamo andati ad utilizzare erano più complicate da utilizzare attraverso il linguaggio Java.
Telegram API - Per consentire ad un operatore di ricevere in tempo reale domande non comprese dal nostro sistema, abbiamo deciso di utilizzare un bot Telegram.
Coqui AI - Per la traduzione da parlato a testo abbiamo utilizzato l'intelligenza artificiale Coqui dato che si tratta di un progetto OPEN ed è possibile addestrarla utilizzando file audio personalizzati.
Django - Django è un framework per le applicazioni web, lo abbiamo scelto per la sua semplicità d'utilizzo e la compatibilità con il linguaggio Python. Questo strumento ci aiuta a gestire le pagine web e la comunicazione con il Database.
PostgreSQL - Abbiamo scelto questo tipo di database dato che è compatibile con Django.
Raspberry pi 3 - Scheda utilizzata per i test, abbiamo utilizzato quella insieme a microfono e casse che avevamo già a casa.
Raspberry + Debian - Come host abbiamo deciso di utilizzare una scheda raspberry con il sistema operativo consigliato, ovvero Debian 11 - Bullseye.

Installazione e requisiti

Per i test abbiamo utilizzato il SO consigliato, Bullseye 32bit standard.
Per l'installazione del SO abbiamo utilizzato imager, un software per scrivere su micro SD (https://www.raspberrypi.com/software/). Prima dell'installazione è possibile inserire dettagli come username e password, connessione ad una specifica rete e connessione SSH.



Altrimenti è possibile modificare il tutto da interfaccia grafica una volta collegato con monitor, tastiera e mouse. Una volta fatto ciò, ci si può connettere tramite SSH.

Come prima cosa vanno eseguiti gli aggiornamenti di sistema con i comandi:
sudo apt update
sudo apt upgrade Tempo necessario circa 15 min

Una volta aggiornato il sistema basta spostare i file del nostro software, per comodità abbiamo utilizzato una chiavetta. Da linea di comando basta utilizzare:
cp -r /media/usb-drive/<nome usb>/CGB %HOME/Desktop/CGB

Una volta raggiunta la cartella dobbiamo installare le librerie necessarie per il software, dato che il raspberry non conterrà altri programmi e non avrà altre funzioni, installiamo senza creare un ambiente virtuale (venv).
python -m pip install -r requirements.txt

La libreria STT legata a Coqui non può essere installata normalmente, serve scaricare la giusta versione in base al SO. Questa è la pagina della versione utilizzata https://github.com/coqui-ai/STT/releases/tag/v1.4.0 , tramite il comando *cat /proc/version* possiamo verificare il tipo di SO in uso, la sigla “cp” utilizzata nelle releases indica la versione di python, nel nostro caso 3.9, una volta trovata la giusta versione è possibile recuperarla tramite:
wget
https://github.com/coqui-ai/STT/releases/download/v1.4.0/stt-1.4.0-cp39-cp39-linux_aarch64.whl

Se sbagliamo versione non sarà avviata l’installazione, quindi se non si è sicuri si può procedere a tentativi, il comando *cat /proc/version* può semplificare la ricerca.
Una volta scaricato il file è possibile installarlo tramite comando:
python -m pip install <nome file>

La gestione delle risposte viene fatta tramite Django e DB quindi è necessario installare postgresql:
sudo apt install postgresql
Quando viene creato imposta un user di default, ovvero **postgres**, si accede a questo user tramite: *sudo su postgres* .
È necessario creare un nuovo utente con lo stesso nome dell’utente impostato nel raspberry, questo perché quando si accede a postgres viene automaticamente utilizzato il nome utente del sistema. In questo caso procederemo con:
createuser aley -P --interactive
Successivamente ci verrà chiesto di impostare una password, questa sarà necessaria nel file **settings.py** per l’interfaccia web e il file **risposte_handler.py** per le richieste al DB; impostare l’utente come **super user**. Fatto ciò, si può creare un DB relativo all’utente appena creato con i comandi:
psql
CREATE DATABASE aley;

Con il comando **psql** si accede all'interfaccia per gestire il DB, il comando help può tornare molto utile.
Per l'esecuzione di file audio in raspberry è necessario installare un ulteriore pacchetto di sistema:
sudo apt-get install libportaudio2

Con questo l'installazione è terminata, ma è possibile che si voglia cambiare Token per il bot telegram o indirizzo IP relativo all'interfaccia web, in quel caso vanno modificati i file *config.txt* e/o il file *mysite/mysite/settings.py* . Nel file settings va cambiato l'accesso al DB in base al nome utente del raspberry e la password inserita precedentemente, idem per *risposte_handler.py*; Inoltre l'ip del dispositivo va aggiunto alla variabile **allowed hosts**.

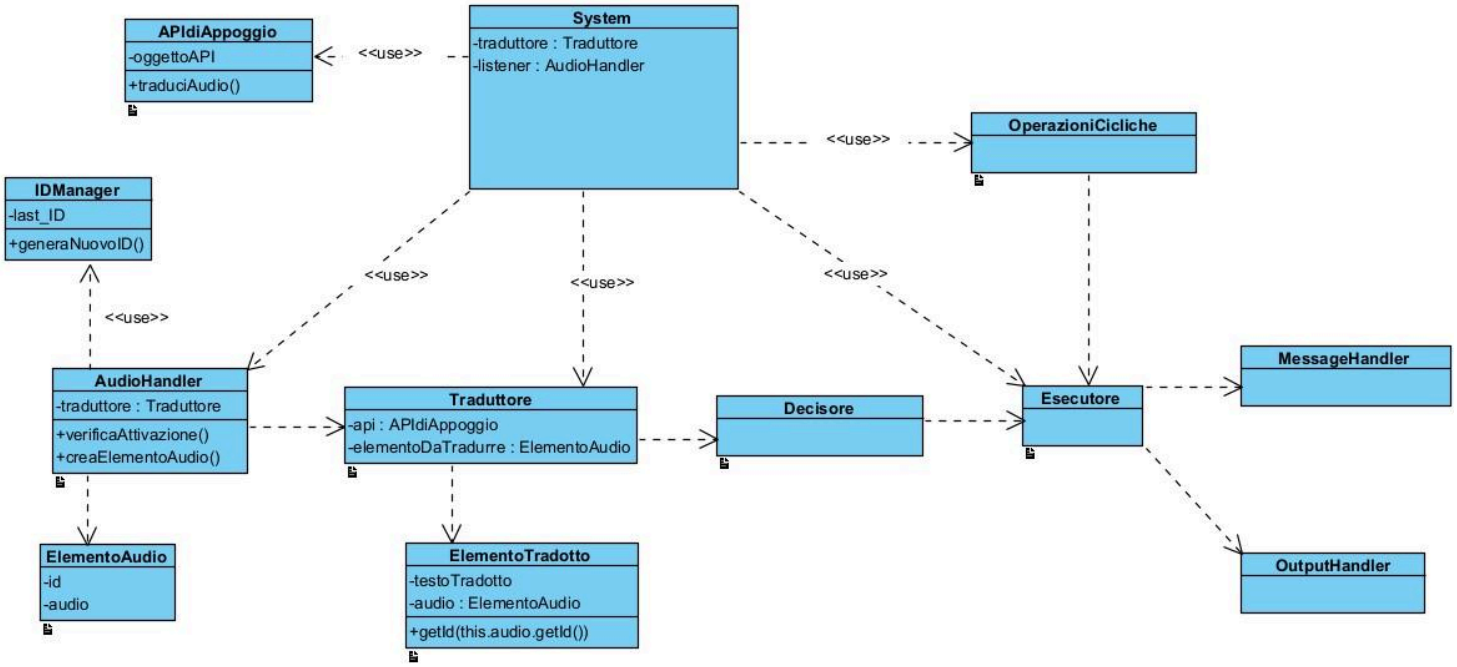
Processo evolutivo del progetto

Nelle prime fasi del progetto abbiamo valutato quali sarebbero potuti essere i principali obiettivi per il nostro bot. La richiesta principale era quella di poter assistere gli anziani rispondendo alle domande che vengono formulate più spesso, semplificando il lavoro degli operatori. Per fare ciò abbiamo ipotizzato l'uso di una API per effettuare la traduzione da parlato a testo, speech to text, e abbiamo valutato diverse opzioni tra cui le API di Google e IBM. Le API di aziende più importanti sarebbero state sicuramente migliori, ma la necessità di pagamenti per servizi più completi e servizi molto ridotti come prova gratuita, abbiamo cercato uno strumento meno restrittivo per effettuare i test preliminari.

Abbiamo trovato una API in linea con le nostre prime necessità, AssemblyAI, che fornisce un servizio gratuito di Speech to Text e un servizio Real-time a pagamento. Abbiamo constatato che l'utilizzo del linguaggio Java avrebbe complicato l'utilizzo della API data la difficoltà nella creazione manuale di richieste HTTP, quindi abbiamo scelto di utilizzare Python per utilizzare direttamente la API, che presentava una documentazione dedicata a questo linguaggio. Un altro punto a favore per Python è quello di essere più flessibile grazie ai suoi moduli, a posteriori ci siamo resi conto che python viene utilizzato molto per l'utilizzo di intelligenze artificiali, semplificando il lavoro nella seconda versione del CareGiverBot. Il servizio gratuito di AseemblyAI non risulta molto veloce, circa 8 secondi tra invio registrazione e download della risposta, ma era sufficiente per le prime fasi di test. La API consente l'invio di un file audio e un ID per distinguere i risultati, quindi nella prima versione abbiamo creato un modulo python per gestirli in maniera più dettagliata nel caso avessimo inviato più file contemporaneamente, ad esempio con più thread.

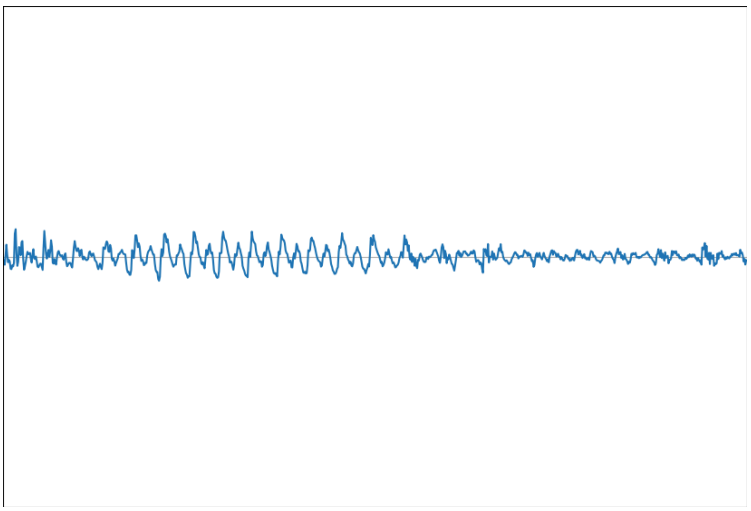
Una volta testato il servizio di AssemblyAI abbiamo creato una bozza di come sarebbe potuta essere la struttura del nostro sistema, quindi abbiamo iniziato a pensare come poter scegliere le risposte in base alle domande ricevute. Dopo

qualche vaga ipotesi abbiamo pensato di scegliere le risposte in base ad un sistema a punti: ogni risposta è formata da alcune parole chiave, per ogni parola nella domanda si valutano le possibili risposte e infine si considera la risposta con più punti. Nel caso due risposte abbiano lo stesso punteggio, allora viene mandato un messaggio all'operatore come se la domanda non fosse stata compresa. Se le risposte vengono inserite con parsimonia, il sistema funziona senza troppi problemi.



Nella prima versione erano state divise le diverse funzionalità tra gli elementi Decisore, Esecutore, System e Traduttore, nella versione successiva abbiamo unito i vari componenti dato che il codice in alcuni casi risultava molto base (esempio: per la traduzione attraverso l'intelligenza artificiale è sufficiente chiamare un metodo).
Per gestire delle domande non comprese abbiamo pensato fin da subito di fornire un feedback a chi si occupa degli anziani, abbiamo scelto di utilizzare Telegram considerata la possibilità di poter programmare un bot personalizzato.

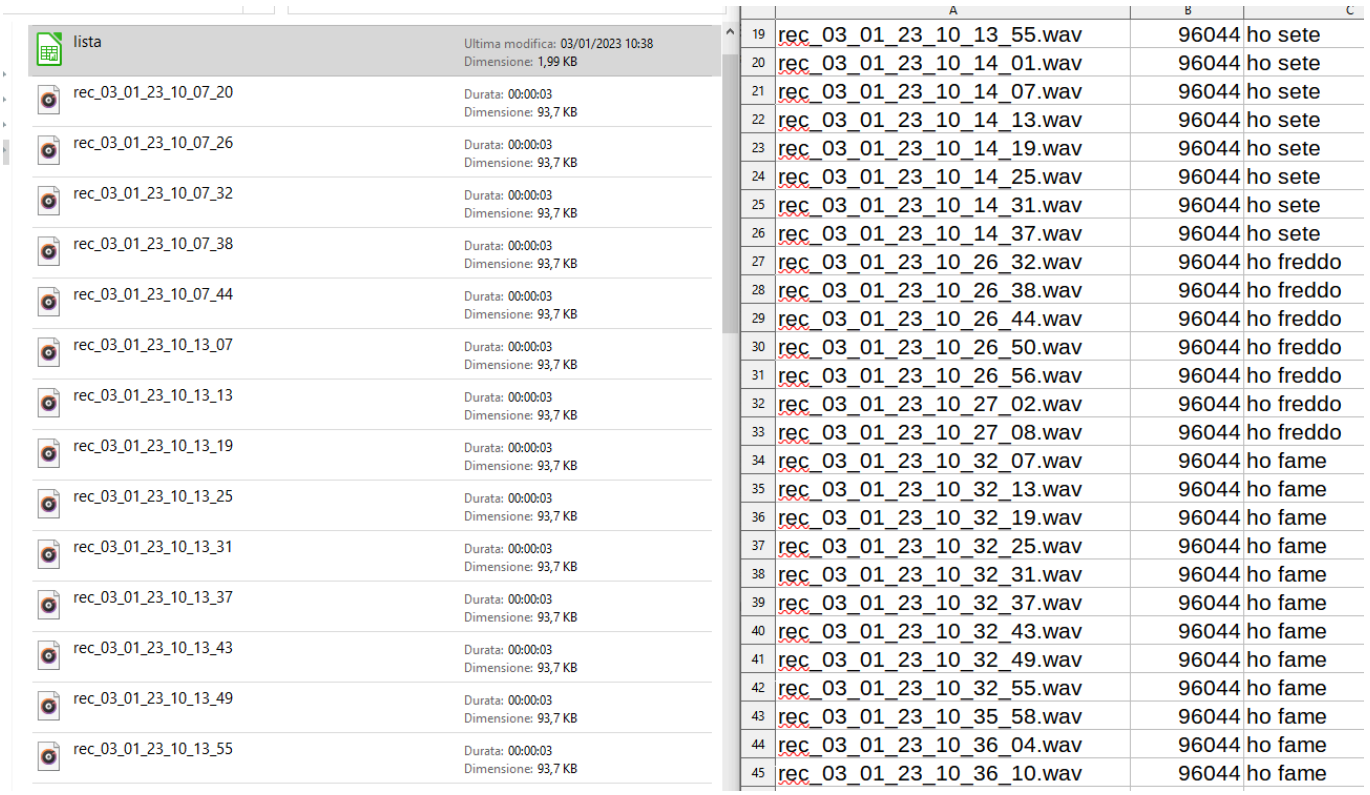
Come prima versione del prototipo consideriamo l'implementazione che prevedeva la traduzione Speech-to-text tramite API, abbiamo abbandonato cambiato strada dato che ci era stato fatta notare che gli anziani non parlano italiano correttamente ma parlano un dialetto che può essere più o meno stretto, e differente in base alla locazione geografica. Al termine di questa prima versione avevamo già iniziato a valutare le opzioni per una interfaccia web, e fatto una bozza dell'algoritmo di scelta della risposta. Inoltre avevamo concluso i test per gestire gli input vocali, l'immagine successiva è uno zoom dei rumori di fondo in un ambiente poco affollato, non studiato per le registrazioni ottimali quindi più simile all'ambiente di utilizzo del CGB.



Nella seconda versione abbiamo sostituito l'API con uno Speech-to-text eseguito da un'intelligenza artificiale. Non avendo conoscenze nell'ambito delle IA, il primo passo è stato quello di farsi un'idea di base per capire meglio di cosa si trattasse e se fosse stata possibile l'implementazione. Tra le varie opzioni OPEN abbiamo individuato Whisper, Wav2Letter++, Mozilla DeepSpeech infine abbiamo scelto di utilizzare CoquiAI dato che la sua documentazione rimane di più semplice comprensione oltre alla presenza di una guida introduttiva piuttosto completa https://stt.readthedocs.io/en/latest/. Per effettuare lo Speech-to-text le IA utilizzano un modello, nella documentazione di Coqui abbiamo trovato gli elementi necessari per crearne uno personalizzato, successivamente abbiamo constatato che tutte le IA OPEN per speech to text utilizzano gli stessi elementi ovvero:

- File CSV contenente 3 colonne
    - nome file registrazione
    - grandezza byte del file
    - testo presente nel file audio
- Insieme delle registrazioni audio
- File Scorer, opzionale, è usato per definire le percentuali di possibilità di utilizzo delle parole.

Per creare il file CSV abbiamo creato uno script Python per la registrazione e scrittura all'interno del file CSV, in modo da: creare il file, calcolare la grandezza in byte e inserire la relativa traduzione in testo; tutto in maniera automatica. Lo script non è stato progettato per un utilizzo finale ma per semplificare la fase di test.



Una volta create una ventina di registrazioni e relativo file CSV abbiamo proceduto a testare la creazione di un modello. Per fare ciò abbiamo creato un container Docker con l'immagine fornita da Coqui, nel quale abbiamo inserito le registrazioni da elaborare, purtroppo non siamo riusciti a creare correttamente un modello. Abbiamo seguito le varie indicazioni ma non avendo conoscenze approfondite della materia, non abbiamo saputo comprendere perfettamente l'origine dell'errore, il comando che abbiamo utilizzato è piuttosto complesso, e anche modificando i suoi valori non abbiamo ottenuto risultati differenti.

```
python -m coqui_stt_training.train --auto_input_dataset lista.csv\
--checkpoint_dir checkpoints \
--checkpoint_secs 1800 \
--export_dir exports \
--plateau_epochs 8 \
--plateau_reduction 0.08 \
--early_stop true \
--es_epochs 18 \
--es_min_delta 0.03 \
--max_to_keep 2 \
--n_hidden 64 \
--dropout_rate 0.3
```

Per non arrestare il processo abbiamo scelto di utilizzare un modello già istruito in lingua italiana, disponibile alla pagina https://github.com/coqui-ai/STT-models.

Una volta confermato l'effettivo funzionamento del sistema di Speech-to-text siamo passati alla realizzazione dell'interfaccia web che avevamo proposto inizialmente, quindi abbiamo iniziato a cercare un framework compatibile con Python. Dopo qualche ricerca abbiamo deciso di utilizzare Django, esso è capace di gestire la nostra interfaccia web e può comunicare con un DB in maniera molto semplificata. Esso consente di gestire le varie pagine web attraverso una divisione di URL, queste pagine corrispondono a Template che sono scritti in HTML, i valori al loro interno possono essere modificati molto facilmente. Le interazioni con il DB vengono gestite da Django tramite dei modelli che rappresentano gli oggetti nel DB, quando si effettuano modifiche è necessario informare il DB per modificare le relative tabelle. Abbiamo diviso le varie pagine in base alla loro funzione e abbiamo lasciato nella Home le funzioni principali ovvero la gestione della sveglia e codice utente di telegram associato all'operatore sanitario. Nel dettaglio abbiamo scelto di limitare al solo id dell'operatore, la ricezione di audio e l'invio delle domande non comprese dal sistema con l'intenzione di evitare che tutti possano accedere alle informazioni che invia il bot. A priori ci siamo resi conto che utilizzare il framework Django ci ha semplificato l'avvio e gestione della componente **server web** ma le sue caratteristiche ci hanno obbligato ad eseguire delle modifiche indesiderate.

Per creare il bot di telegram è stato necessario interagire con un bot ufficiale chiamato BotFather, il quale fornisce una chiave associata al bot che viene creato. La chiave è necessaria per interagire con il servizio fornito da Telegram e poter inviare/ricevere comandi, messaggi, file etc.. Lo scopo del bot è specificare all'utente il suo ID (così da inserirlo nel sistema tramite l'interfaccia web), ricevere file audio o messaggi vocali ma anche inviare all'utente collegato la domanda che non viene compresa, sotto forma di file audio.

Una volta completati tutti i componenti singoli abbiamo iniziato a testare la loro esecuzione combinata, purtroppo abbiamo avuto alcuni conflitti legati alla posizione di alcuni file o cartelle, quindi per fixare abbiamo effettuato qualche modifica non troppo elegante.

Confermato il corretto funzionamento del sistema (ad eccezione di telegram) siamo passati ai test per la scheda Raspberry. Inizialmente abbiamo installato il SO consigliato Debian Bullseye, abbiamo spostato i file necessari e installato le librerie necessarie. Abbiamo installato PostgreSQL come DB e una volta installato abbiamo modificato le impostazioni di Django per utilizzare quel DB e l'indirizzo IP corrispondente al Raspberry nella rete locale. Una volta preparato il tutto abbiamo testato i vari componenti, purtroppo l'esecuzione dell'elemento principale, ovvero la gestione dell'audio e traduzione in testo, è risultata molto più lenta del previsto. Utilizzando un modello ITA da soli 45MB il tempo necessario per lo Speech-to-text è di 4-5 secondi minimo, e l'esecuzione dopo la prima traduzione diventa instabile; per tentare di migliorare le prestazioni abbiamo proposto qualche soluzione secondaria, nel frattempo abbiamo provato ad utilizzare un SO più leggero senza interfaccia

grafica. Il SO Bullseye Lite 32Bit a prima vista rimane più leggero, ma abbiamo notato fin da subito la mancanza di elementi base come la gestione automatica di chiavi USB o come i moduli Python **pip**, utilizzato per le installazioni di altri moduli, e **venv**, utilizzato per la creazione di ambienti virtuali Python.

Possibili migliorie

Il funzionamento del CGB con il codice attuale lascia molto a desiderare, abbiamo considerato che deve gestire molti componenti quindi potrebbe essere di aiuto un hardware più performante. In alternativa abbiamo pensato che per alleggerire il sistema si potrebbero dividere i componenti ed eseguirli tra più elementi. Più nel dettaglio abbiamo ipotizzato un'architettura del tipo client-server in cui la scheda raspberry si limita ad ascoltare e riprodurre audio mentre un server locale si occupa di gestire le domande, risposte, configurazioni e quant'altro. Ovviamente questa ipotesi vanifica lo scopo di rendere tutto economico, poco ingombrante e semplice, ma potrebbe servire come spunto per una visione differente del sistema.

Considerazioni finali

Il CGB è lontano dall'essere un prodotto perfetto e ci sono sicuramente delle migliorie che possono essere effettuate.
Dal punto di vista tecnico un migliore metodo di rilevazione audio aiuterebbe a filtrare parole che non sono rivolte al CGB, a differenza dalla registrazione continua attualmente in uso che registra qualsiasi cosa il bot ascolta, non per forza domande rivolte ad esso. Un metodo simile al richiamo del bot come "hey alexa" o "ok google" è stato sconsigliato perché un anziano potrebbe avere problemi ad assimilare il funzionamento.
Un altro problema da affrontare è che molti anziani non parlano un italiano corretto, ma principalmente in dialetto e il CGB non è istruito nel tradurre espressioni dialettali, questo rende molto difficile per lui filtrare le parole chiave da associare alle risposte nel suo database. Inoltre sarebbero necessari più modelli Speech-to-text in base al dialetto utilizzato in una determinata struttura.
Nonostante queste problematiche siamo dell'idea che il CGB sia una buona base da cui partire per creare un sistema in grado di migliorare le vite degli anziani che ne faranno uso.